

Movimiento lateral de un atacante como flujo gradiente

Una perspectiva vectorial de la ciberseguridad ofensiva
con validación en laboratorio Wazuh + Suricata en tiempo real

Integrantes: Daniel Pareja Franco
Santiago José Curi
Sebastián Páez
Nicole Bonilla

Asignatura: Cálculo Vectorial

Campo elegido: Ciberseguridad ofensiva — movimiento lateral (MITRE TA0008)

Herramienta computacional: Python 3.11 · NumPy · SciPy · Matplotlib · Wazuh 4.7.5

Infraestructura: DigitalOcean (3 VMs Ubuntu) · Kali Linux · Suricata IDS

Fecha: Mayo de 2026

Declaración metodológica. Este proyecto combina un modelo analítico cerrado (Fase 1) con un laboratorio real en la nube (Fase 2). En la Fase 1, $V(x, y)$ se construye de forma explícita y todos los resultados son exactos. En la Fase 2, V se alimenta de alertas reales de Wazuh y Suricata, haciendo que el campo escalar evolucione en tiempo real durante el ataque simulado. Todos los scripts, configuraciones y datos del experimento están disponibles en el repositorio del proyecto.

Índice

1. Planteamiento del problema	4
2. Infraestructura del laboratorio	4
2.1. Wazuh SIEM	4
2.2. Suricata IDS	5
2.3. Estructura de alertas y datos extraídos	5
2.4. Topología del laboratorio	5
3. Motor de gradiente en Python	6
3.1. Arquitectura del sistema	6
3.2. Módulo <code>wazuh_client.py</code> — Adquisición y puntuación	6
3.3. Módulo <code>scalar_field.py</code> — Campo escalar dinámico	7
3.4. Módulo <code>gradient_flow.py</code> — Trayectoria y integral	8
3.5. Módulo <code>visualizer.py</code> — Figura de cuatro paneles	8
3.6. Loop principal: <code>main.py</code>	9
4. Conceptos de cálculo vectorial utilizados	9
5. Modelo matemático	9
5.1. Fase 1 — Modelo analítico (caso base)	9
5.1.1. Red y campo escalar	9
5.1.2. Gradiente	10
5.1.3. Gradiente evaluado en los nodos	10
5.1.4. Trayectoria del atacante	10
5.1.5. Punto crítico y Hessiano	11
5.1.6. Conservatividad e integral de línea	11
5.1.7. Irrotacionalidad	11
5.2. Fase 2 — Modelo dinámico con Wazuh	11
6. Metodología experimental	11
6.1. Parámetros de configuración	11
6.2. Protocolo	12
6.3. Simulador de ataque (<code>attack_sim.py</code>)	12
6.4. Motor en tiempo real	14
6.5. Alertas registradas en Wazuh — <code>victim-c</code>	15
6.6. Clasificación MITRE ATT&CK	15
7. Resultados y análisis	16
7.1. Evolución del campo escalar durante el experimento	16
7.2. Observación 1 — V responde en tiempo real	17
7.3. Observación 2 — El saddle point es el hallazgo topológico central	18
7.4. Observación 3 — Puntos críticos correlacionan con estructura del campo	18
7.5. Observación 4 — Tres nodos activos colapsan la integral	18
7.6. Observación 5 — La trayectoria $A \rightarrow B \rightarrow C$ valida el movimiento lateral	18

8. Discusión: alcances, limitaciones y posibles mejoras	19
8.1. Alcances	19
8.2. Limitaciones	19
8.3. Posibles mejoras	19
9. Bitácora de uso de inteligencia artificial	19
10. Conclusiones	20
11. Referencias bibliográficas	21
Anexos: Evidencia fotográfica complementaria	22

1. Planteamiento del problema

En un ataque informático dirigido, el adversario rara vez compromete directamente su objetivo final. Primero obtiene acceso a un nodo de baja seguridad y luego se **mueve lateralmente** por la red saltando de equipo en equipo hasta alcanzar el activo de mayor valor: una base de datos, un servidor de control o credenciales administrativas. Este proceso no es aleatorio: un atacante con información sobre la red elige en cada paso el nodo que maximiza su acceso futuro con el menor riesgo de detección. Esa toma de decisiones tiene una estructura matemática precisa.

Pregunta central del proyecto

¿Puede el gradiente de un campo escalar de vulnerabilidad modelar la trayectoria óptima de un atacante en movimiento lateral, y qué revela ese modelo sobre dónde concentrar las defensas?

2. Infraestructura del laboratorio

2.1. Wazuh SIEM

Wazuh es una plataforma de código abierto para *Security Information and Event Management* (SIEM). Recopila, normaliza y correlaciona eventos de seguridad de múltiples fuentes en tiempo real.

La arquitectura consta de tres componentes principales:

- **Agentes:** procesos ligeros instalados en cada host monitorizado. Recolectan logs del sistema operativo (syslog, auth.log, dpkg), ejecutan comprobaciones de integridad de archivos (FIM) y envían todo al manager vía canal cifrado.
- **Manager:** servidor central que recibe los eventos de los agentes, aplica el motor de reglas y genera las alertas. Las alertas se almacenan en `/var/ossec/logs/alerts/alerts.json` como objetos JSON, uno por línea.
- **Dashboard:** interfaz web que consume el índice de alertas almacenado en OpenSearch (heredero de Elasticsearch).

Cada alerta generada por Wazuh incluye, entre otros campos:

Cuadro 1: Campos relevantes de una alerta Wazuh (extracto del JSON).

Campo	Tipo	Descripción
timestamp	string ISO 8601	Momento del evento
agent.id	string (3 dígitos)	Identificador del agente origen
agent.name	string	Hostname del agente
rule.id	entero	ID de la regla que disparó la alerta
rule.level	entero (1–15)	Severidad: ≤ 4 info, 5–7 bajo, 8–10 alto, ≥ 11 crítico
rule.description	string	Descripción legible de la alerta
rule.mitre.tactic	lista	Táctica(s) MITRE ATT&CK asociada(s)
rule.mitre.id	lista	Técnica(s) MITRE (e.g. T1110.001)

Nota técnica. Wazuh 4.7.5 no expone un endpoint REST `/alerts` (fue eliminado en versiones post-4.3). El motor de gradiente accede a las alertas directamente vía SSH leyendo `alerts.json` con `tail -n 5000`, filtrando por `agent.id` y ventana temporal.

2.2. Suricata IDS

Suricata es un motor de detección de intrusiones de red (*Network IDS/IPS*) de código abierto. Analiza el tráfico de red a nivel de paquete y dispara alertas cuando detecta patrones maliciosos definidos en sus reglas.

En el laboratorio, Suricata corre en el manager y monitoriza el tráfico entrante hacia los nodos víctima. Las reglas provienen del conjunto **Emerging Threats** (ET), que incluye:

- **ET DROP:** bloqueo de IPs en listas negras (Dshield, CINS). Rule ID 86601.
- **ET CINS:** IPs con reputación baja según *Active Threat Intelligence*. Rule ID 86601.
- **Alertas de autenticación SSH:** detectadas por el agente Wazuh a través del decoder de `sshd`. Rule ID 5710 (*sshd: Attempt to login using a non-existent user*), clasificadas como T1110.001 (Brute Force: Password Guessing) + T1021.004 (Remote Services: SSH) en el marco MITRE ATT&CK.

2.3. Estructura de alertas y datos extraídos

El motor extrae únicamente alertas de `rule.level ≥ 5` para filtrar ruido operacional (heartbeats, syscollector, etc.). A partir de esas alertas calcula el score de vulnerabilidad de cada nodo con una fórmula de decaimiento temporal exponencial (véase sección 5).

2.4. Topología del laboratorio

Tres máquinas virtuales Ubuntu en DigitalOcean actúan como nodos de la red simulada. El atacante opera desde Kali Linux en red local accediendo a los nodos por IP pública.

Cuadro 2: Nodos del laboratorio y su rol en el experimento.

Rol	Hostname	IP pública	Wazuh Agent ID	Pos. lógica
Nodo A (entry)	wazuh-manager	165.227.109.227	000 (manager)	(0.5, 0.5)
Nodo B (víctima)	victim-c	165.227.122.79	003	(3.0, 3.0)
Nodo C (crown jewel)	capstone-campusshield	198.199.71.219	001	(7.0, 3.0)
Atacante	Kali Linux	192.168.50.21	—	—

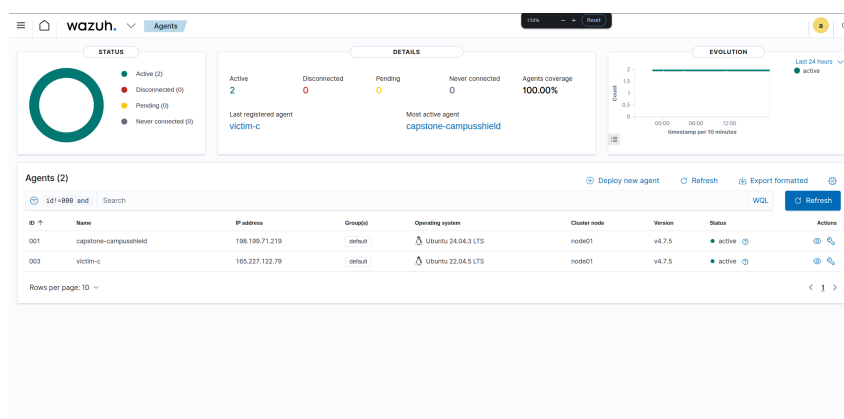


Figura 1: Vista de agentes en el dashboard de Wazuh. Los dos agentes activos son capstone-campusshield (001) y victim-c (003), ambos ejecutando Wazuh v4.7.5. La cobertura es del 100 %.

3. Motor de gradiente en Python

3.1. Arquitectura del sistema

El motor está compuesto por cinco módulos Python que se comunican a través de la función principal en `main.py`. El flujo de datos en cada iteración es:

Wazuh (SSH) → wazuh_client → scalar_field → gradient_flow → visualizer

Cada iteración dura aproximadamente 5 segundos (`POLL_INTERVAL`). Al finalizar, la figura generada se guarda en `/tmp/` y se transfiere automáticamente vía SCP a la máquina local del analista.

3.2. Módulo `wazuh_client.py` — Adquisición y puntuación

Este módulo reemplaza la comunicación REST de Wazuh con acceso directo vía SSH al archivo de alertas. Contiene dos funciones principales:

Listing 1: Lectura de alertas vía SSH y filtrado temporal.

```

1 def get_alerts_for_agent(url, token, agent_id, last_n_minutes=3):
2     since = datetime.now(timezone.utc) - timedelta(minutes=
3         last_n_minutes)
4     # Conexion SSH al manager y lectura de las ultimas 5000 lineas
5     result = subprocess.run(
6         ["ssh", "-o", "BatchMode=yes", f"root@165.227.109.227",
7         "tail -n 5000 /var/ossec/logs/alerts/alerts.json"],
8         capture_output=True, text=True, timeout=12)
9     # Filtrar por agente y ventana temporal
10    for line in result.stdout.splitlines():
11        alert = json.loads(line)
12        if alert["agent"]["id"].zfill(3) != agent_id.zfill(3):
13            continue
14        if datetime.fromisoformat(alert["timestamp"]) < since:
15            continue
16        alerts.append(alert)
17    return alerts

```

Listing 2: Cálculo del score con decaimiento temporal y normalización logarítmica.

```

1 def score_from_alerts(alerts, decay_lambda=0.1):
2     # Solo alertas nivel >= 5 (descartar ruido operacional)
3     for alert in alerts:
4         level = alert["rule"]["level"]
5         if level < 5: continue
6         delta_min = (now - ts).total_seconds() / 60
7         # Peso: nivel de alerta * factor de decaimiento exponencial
8         total += level * exp(-decay_lambda * delta_min)
9     # Normalizacion log1p para evitar saturacion con muchas alertas
10    score = log1p(total) / log1p(50) * 10.0
11    return min(score, 10.0)

```

La normalización $\ln(1+x)/\ln(51)$ hace que el score crezca rápidamente al inicio pero se aplane con volúmenes muy altos de alertas, evitando que todos los nodos queden saturados en 10.0.

3.3. Módulo `scalar_field.py` — Campo escalar dinámico

Construye $V(\mathbf{x}, t)$ como superposición de gaussianas centradas en los nodos, con amplitudes iguales a los scores actuales. El parámetro $\sigma = 1.5$ define el **radio de influencia** de cada nodo en unidades lógicas: a distancia σ del nodo, la contribución gaussiana cae a $e^{-0.5} \approx 60\%$ del máximo; a distancia 2σ cae a $e^{-2} \approx 14\%$. Un σ más grande «ensancha» los picos y suaviza el campo; uno más pequeño crea picos más estrechos con gradientes más abruptos.

Listing 3: Construcción del campo escalar por superposición gaussiana.

```

1 def build_scalar_field(node_positions, scores, sigma, grid_size=200):
2     X, Y = np.meshgrid(np.linspace(-2, 10, grid_size),
3                        np.linspace(-2, 7, grid_size))
4     V = np.zeros_like(X)
5     for (px, py), s in zip(node_positions, scores):
6         dist2 = (X - px)**2 + (Y - py)**2
7         V += s * np.exp(-dist2 / (2 * sigma**2))
8     return X, Y, V
9
10 def compute_gradient(X, Y, V):
11     # Gradiente numerico con diferencias finitas (numpy.gradient)
12     dVdy, dVdx = np.gradient(V, Y[:,0], X[0,:])
13     return dVdx, dVdy
14
15 def find_critical_points(X, Y, dVdx, dVdy, threshold=0.02):
16     # Puntos donde |gradiente| < umbral -> candidatos a puntos
17     # criticos
18     mag = np.sqrt(dVdx**2 + dVdy**2)
19     mask = mag < threshold
20     # Agrupar puntos adyacentes con label de scipy
21     ...

```

3.4. Módulo `gradient_flow.py` — Trayectoria y integral

¿Por qué dos etapas de integración? El flujo gradiente $\mathbf{r}'(t) = \nabla V(\mathbf{r}(t))$ converge siempre hacia el *máximo local más cercano* al punto de partida. Cuando victim-c (B) y capstone (C) están ambos activos, B actúa como cuenca de atracción para puntos que parten desde el área de A. La trayectoria se «detiene» en B porque $\nabla V \approx \mathbf{0}$ en un máximo. Para modelar el salto B→C, se reinicia la integración desde el último punto de la etapa anterior, replicando la decisión del atacante de moverse al siguiente objetivo una vez comprometido B.

Integra la EDO $\mathbf{r}'(t) = \nabla V(\mathbf{r}(t))$ con el solver RK45 de SciPy, que adapta el paso de integración automáticamente:

Listing 4: Integración numérica del flujo gradiente con RK45.

```

1 def integrate_attacker_path(X, Y, dVdx, dVdy, start, t_span, n_steps)
2     :
3     # Interpolacion bilineal del gradiente en puntos arbitrarios
4     interp_x = RegularGridInterpolator((Y[:,0], X[0,:]), dVdx)
5     interp_y = RegularGridInterpolator((Y[:,0], X[0,:]), dVdy)
6
7     def rhs(t, r): # Lado derecho de la EDO
8         return [interp_x([r[1], r[0]])[0],
9                 interp_y([r[1], r[0]])[0]]
10
11     sol = solve_ivp(rhs, t_span, start, method='RK45',
12                    t_eval=np.linspace(*t_span, n_steps))
13     return np.array([sol.y[0], sol.y[1]]) # path[0]=x, path[1]=y
14
15 def compute_line_integral(X, Y, V, path):
16     # TF de integrales de linea: V(destino) - V(origen)
17     interp_V = RegularGridInterpolator((Y[:,0], X[0,:]), V)
18     v_end = interp_V([[path[1,-1], path[0,-1]]])[0]
19     v_start = interp_V([[path[1,0], path[0,0]]])[0]
20     return float(v_end - v_start)

```

3.5. Módulo `visualizer.py` — Figura de cuatro paneles

Genera en cada iteración una figura Matplotlib con fondo oscuro compuesta por cuatro paneles:

1. **Campo escalar $V(\mathbf{x})$:** mapa de calor con isolíneas. Las zonas más rojas corresponden a nodos con mayor actividad de alertas.
2. **Gradiente ∇V :** campo vectorial quiver sobre el heatmap semitransparente. Las flechas cyan apuntan hacia los máximos del campo.
3. **Trayectoria $\mathbf{r}(t)$:** integración en dos etapas — línea azul sólida (A→B) y naranja punteada (B→C). Marcadores: círculo verde (entrada), triángulo amarillo (pivote B), estrella roja (destino C).
4. **Panel numérico:** iteración actual, integral de línea, número de puntos críticos y scores por nodo.

3.6. Loop principal: `main.py`

El loop principal ejecuta en cada iteración:

1. Consulta las alertas de los tres agentes vía SSH
2. Calcula los scores con decaimiento temporal
3. Construye $V(x, y)$ y calcula ∇V numéricamente
4. Integra la trayectoria en dos etapas ($A \rightarrow B$ y $B \rightarrow C$)
5. Calcula la integral de línea y detecta puntos críticos
6. Actualiza la figura en pantalla y la guarda en `/tmp/`
7. Transfiere la figura vía SCP al equipo local del analista

Lo que se espera del motor: a medida que el simulador de ataque genera alertas SSH en los nodos víctima, los picos gaussianos de V deben crecer, el gradiente debe orientarse hacia esos picos, y la trayectoria $\mathbf{r}(t)$ debe mostrar el camino $A \rightarrow B \rightarrow C$ de forma progresiva.

4. Conceptos de cálculo vectorial utilizados

Cuadro 3: Conceptos del curso y su rol en el modelo.

Concepto	Rol en el modelo
Campo escalar $V(\mathbf{x})$	Función de vulnerabilidad definida sobre el espacio lógico de la red
Gradiente ∇V	Dirección de mayor vulnerabilidad en cada punto — guía el ataque
Curva parametrizada $\mathbf{r}(t)$	Trayectoria del atacante como solución de la EDO $\mathbf{r}' = \nabla V$
Integral de línea $\int_C \nabla V \cdot d\mathbf{r}$	Vulnerabilidad acumulada a lo largo del recorrido
Campo conservativo	∇V es conservativo — solo importan origen y destino
Puntos críticos de V	Máximos = activos críticos; sillars = cuellos de botella defensivos
Rotacional $\nabla \times \nabla V = \mathbf{0}$	El campo gradiente es siempre irrotacional

5. Modelo matemático

5.1. Fase 1 — Modelo analítico (caso base)

5.1.1. Red y campo escalar

Se modela una red con 6 activos en el plano lógico $\mathbb{R}^2$. Las coordenadas no son físicas sino abstractas: representan la cercanía lógica de cada nodo al núcleo de la red y su nivel de privilegio.

Cuadro 4: Nodos de la red, coordenadas lógicas y vulnerabilidad evaluada.

Nodo	Activo	Coordenadas (x, y)	$V(x, y)$	Rol
A	Equipo de usuario	(0, 0)	2.0	Punto de entrada
B	Servidor de archivos	(2, 1)	33.1	Pivote intermedio
C	Servidor de aplicaciones	(3, 3)	47.9	Pivote intermedio
D	Controlador de dominio	(5, 2)	60.0	Alto privilegio
E	Base de datos	(7, 3)	67.1	Crown jewel
F	Servidor de respaldo	(4, -1)	37.7	Objetivo secundario

El campo escalar de vulnerabilidad es:

$$V(x, y) = 60 - (x - 7)^2 - (y - 3)^2 + 0.8x + 0.5y \quad (1)$$

La parte cuadrática negativa garantiza un máximo único cerca del nodo E (7, 3). Los términos lineales modelan el aumento gradual de exposición al avanzar hacia el interior de la red.

5.1.2. Gradiente

$$\nabla V(x, y) = (14.8 - 2x, 6.5 - 2y) \quad (2)$$

5.1.3. Gradiente evaluado en los nodos

Cuadro 5: Dirección del ataque: ∇V evaluado en cada nodo.

Nodo	Coordenadas	$\nabla V = (14.8 - 2x, 6.5 - 2y)$	Interpretación
A	(0, 0)	(14.8, 6.5)	Fuerte impulso hacia el interior
B	(2, 1)	(10.8, 4.5)	Avanza hacia zonas de mayor privilegio
C	(3, 3)	(8.8, 0.5)	Movimiento principalmente horizontal
D	(5, 2)	(4.8, 2.5)	Se acerca al crown jewel
E	(7, 3)	(0.8, 0.5)	Gradiente casi nulo — máxima vulnerabilidad

5.1.4. Trayectoria del atacante

Siguiendo el campo gradiente, $\mathbf{r}'(t) = \nabla V(\mathbf{r}(t))$, se obtiene el sistema de EDOs:

$$x'(t) = 14.8 - 2x(t), \quad y'(t) = 6.5 - 2y(t) \quad (3)$$

Cada ecuación es separable. Para $x(t)$:

$$\frac{dx}{14.8 - 2x} = dt \implies -\frac{1}{2} \ln |14.8 - 2x| = t + C_1 \implies x(t) = 7.4 - Ae^{-2t} \quad (4)$$

Con $x(0) = 0$: $A = 7.4$. Análogamente $y(t) = 3.25(1 - e^{-2t})$. La solución es:

con $\mathbf{r}(0) = (0, 0)$:

$$\boxed{\mathbf{r}(t) = (7.4(1 - e^{-2t}), 3.25(1 - e^{-2t}))} \quad (5)$$

Converge a $(7.4, 3.25) \approx E$ cuando $t \rightarrow \infty$.

5.1.5. Punto crítico y Hessiano

El único punto crítico, $\nabla V = \mathbf{0}$, es $(7.4, 3.25)$ con Hessiano:

$$H_V = \begin{pmatrix} -2 & 0 \\ 0 & -2 \end{pmatrix} \quad (6)$$

H_V es negativa definida ($\det H_V = 4 > 0$): **máximo local**.

5.1.6. Conservatividad e integral de línea

$\partial P/\partial y = \partial Q/\partial x = 0$ confirma que ∇V es **conservativo**. Por el Teorema Fundamental:

$$\int_C \nabla V \cdot d\mathbf{r} = V(7, 3) - V(0, 0) = 67.1 - 2.0 = \mathbf{65.1} \quad (7)$$

5.1.7. Irrotacionalidad

$\nabla \times (\nabla V) = \mathbf{0}$: no existen «remolinos» de vulnerabilidad; un camino cerrado no acumula exposición neta.

5.2. Fase 2 — Modelo dinámico con Wazuh

En el laboratorio V emerge de datos reales:

$$V(\mathbf{x}, t) = \sum_{i=1}^k v_i(t) \cdot \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{2\sigma^2}\right) \quad (8)$$

$$v_i(t) = \frac{\ln\left(1 + \sum_j \text{level}_j \cdot e^{-\lambda(t-t_j)}\right)}{\ln(51)} \times 10 \quad \in [0, 10] \quad (9)$$

Las alertas recientes pesan más; alertas antiguas dentro de la ventana de 3 minutos decaen exponencialmente.

6. Metodología experimental

6.1. Parámetros de configuración

Cuadro 6: Parámetros del motor de gradiente durante el experimento.

Parámetro	Valor	Descripción
SIGMA	1.5	Radio de influencia gaussiana (unidades lógicas)
ALERT_WINDOW	3 min	Ventana temporal para contar alertas activas
DECAY_LAMBDA	0.1 min^{-1}	Tasa de decaimiento del peso de alertas
POLL_INTERVAL	5 s	Intervalo entre iteraciones del motor
GRID_SIZE	200×200	Resolución de la malla espacial
ENTRY_OFFSET	(0.5, 0.5)	Desplazamiento del punto de entrada respecto a A

6.2. Protocolo

El experimento se ejecutó en una única sesión de laboratorio con las siguientes fases en paralelo:

1. **Terminal 1** (Kali): ejecución del motor de gradiente (`main.py`), que monitoriza y visualiza en tiempo real.
2. **Terminal 2** (Kali): ejecución del simulador de ataque (`attack_sim.py`), que genera las alertas SSH.
3. **Navegador**: dashboard Wazuh abierto para verificación visual de las alertas recibidas.

El experimento duró **53 iteraciones** (≈ 265 segundos). El motor guardó y transfirió automáticamente una figura por iteración.

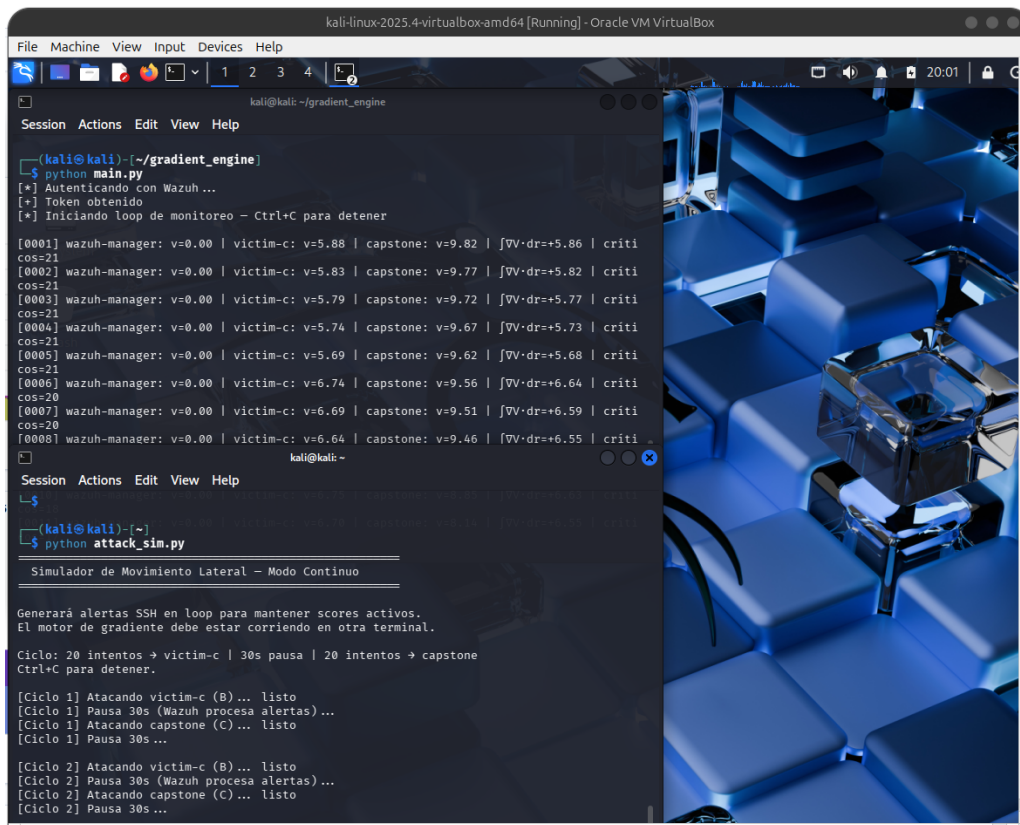
6.3. Simulador de ataque (`attack_sim.py`)

El simulador ejecuta un ciclo continuo de intentos de autenticación SSH con credenciales comunes (`root`, `admin`, `ubuntu`, etc.) usando `sshpass`. El ciclo alterna entre los dos nodos víctima con pausas de 30 segundos para que Wazuh procese las alertas:

20 intentos $\rightarrow$ **victim-c** $\rightarrow$ pausa 30s $\rightarrow$ **20 intentos** $\rightarrow$ **capstone** $\rightarrow$ pausa 30s $\rightarrow$ repite

Cada intento usa `-o PasswordAuthentication=yes` para forzar autenticación por contraseña (sin clave pública), lo que genera alertas `sshd` de nivel 5 en Wazuh (Rule 5710).

Configuración previa necesaria. Por defecto, capstone tenía `PasswordAuthentication` no en su configuración SSH, lo que impedía que los intentos de brute-force generaran alertas de nivel 5. Fue necesario habilitarlo en `/etc/ssh/sshd_config` y reiniciar el servicio para que los intentos fallidos quedaran registrados como Rule 5710 en Wazuh. Sin este paso, capstone permanecía con score $v_C = 0$ aunque recibiera intentos de conexión.



```
kali@kali: ~/gradient_engine
$ python main.py
[*] Autenticando con Wazuh ...
[*] Token obtenido
[*] Iniciando loop de monitoreo - Ctrl+C para detener

[0001] wazuh-manager: v=0.00 | victim-c: v=5.88 | capstone: v=9.82 | ∫V·dr=+5.86 | criti
cos=21
[0002] wazuh-manager: v=0.00 | victim-c: v=5.83 | capstone: v=9.77 | ∫V·dr=+5.82 | criti
cos=21
[0003] wazuh-manager: v=0.00 | victim-c: v=5.79 | capstone: v=9.72 | ∫V·dr=+5.77 | criti
cos=21
[0004] wazuh-manager: v=0.00 | victim-c: v=5.74 | capstone: v=9.67 | ∫V·dr=+5.73 | criti
cos=21
[0005] wazuh-manager: v=0.00 | victim-c: v=5.69 | capstone: v=9.62 | ∫V·dr=+5.68 | criti
cos=21
[0006] wazuh-manager: v=0.00 | victim-c: v=6.74 | capstone: v=9.56 | ∫V·dr=+6.64 | criti
cos=20
[0007] wazuh-manager: v=0.00 | victim-c: v=6.69 | capstone: v=9.51 | ∫V·dr=+6.59 | criti
cos=20
[0008] wazuh-manager: v=0.00 | victim-c: v=6.64 | capstone: v=9.46 | ∫V·dr=+6.55 | criti
cos=20

kali@kali: ~
$ python attack_sim.py

Simulador de Movimiento Lateral - Modo Continuo

Generará alertas SSH en loop para mantener scores activos.
El motor de gradiente debe estar corriendo en otra terminal.

Ciclo: 20 intentos → victim-c | 30s pausa | 20 intentos → capstone
Ctrl+C para detener.

[Ciclo 1] Atacando victim-c (B)... listo
[Ciclo 1] Pausa 30s (Wazuh procesa alertas)...
[Ciclo 1] Atacando capstone (C)... listo
[Ciclo 1] Pausa 30s ...

[Ciclo 2] Atacando victim-c (B)... listo
[Ciclo 2] Pausa 30s (Wazuh procesa alertas)...
[Ciclo 2] Atacando capstone (C)... listo
[Ciclo 2] Pausa 30s ...
```

Figura 2: Terminal de Kali con `attack_sim.py` en ejecución. Se observan los ciclos de ataque: 20 intentos hacia victim-c, pausa de 30s para procesamiento de Wazuh, y 20 intentos hacia capstone. El contador de ciclos indica que el simulador ya lleva varios minutos activo.

6.4. Motor en tiempo real

```

kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
kali@kali: ~/gradient_engine
Session Actions Edit View Help
cos=9
[0040] wazuh-manager: v=4.49 | victim-c: v=7.90 | capstone: v=9.38 |  $\int \nabla V \cdot d\mathbf{r}=+0.15$  | criti
cos=7
[0041] wazuh-manager: v=4.44 | victim-c: v=7.38 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.16$  | crit
icos=8
[0042] wazuh-manager: v=4.40 | victim-c: v=7.95 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.14$  | crit
icos=8
[0043] wazuh-manager: v=4.36 | victim-c: v=7.91 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.14$  | crit
icos=7
[0044] wazuh-manager: v=4.32 | victim-c: v=8.80 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.10$  | crit
icos=8
[0045] wazuh-manager: v=4.27 | victim-c: v=8.75 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.10$  | crit
icos=8
[0046] wazuh-manager: v=4.23 | victim-c: v=8.00 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.12$  | crit
icos=8
[0047] wazuh-manager: v=4.18 | victim-c: v=7.95 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.12$  | crit
icos=8
[0048] wazuh-manager: v=4.14 | victim-c: v=7.89 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.12$  | crit
icos=7
[0049] wazuh-manager: v=4.09 | victim-c: v=7.83 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.11$  | crit
icos=7
[0050] wazuh-manager: v=4.04 | victim-c: v=8.74 | capstone: v=10.00 |  $\int \nabla V \cdot d\mathbf{r}=+0.08$  | crit

kali@kali: ~
Session Actions Edit View Help
Ctrl+C para detener.

[Ciclo 1] Atacando victim-c (B)... listo
[Ciclo 1] Pausa 30s (Wazuh procesa alertas)...
[Ciclo 1] Atacando capstone (C)... listo
[Ciclo 1] Pausa 30s ...

[Ciclo 2] Atacando victim-c (B)... listo
[Ciclo 2] Pausa 30s (Wazuh procesa alertas)...
[Ciclo 2] Atacando capstone (C)... listo
[Ciclo 2] Pausa 30s ...

[Ciclo 3] Atacando victim-c (B)... listo
[Ciclo 3] Pausa 30s (Wazuh procesa alertas)...
[Ciclo 3] Atacando capstone (C)... listo
[Ciclo 3] Pausa 30s ...

[Ciclo 4] Atacando victim-c (B)... listo
[Ciclo 4] Pausa 30s (Wazuh procesa alertas)...
[Ciclo 4] Atacando capstone (C)... listo
[Ciclo 4] Pausa 30s ...

[Ciclo 5] Atacando victim-c (B)... listo
[Ciclo 5] Pausa 30s (Wazuh procesa alertas)...

```

Figura 3: Terminal de Kali con `main.py` mostrando la salida por consola del motor de gradiente. Cada línea corresponde a una iteración e incluye: número de iteración, score de cada nodo (v_A , v_B , v_C) e integral de línea $\int \nabla V \cdot d\mathbf{r}$. Se aprecia cómo los scores de victim-c y capstone crecen a medida que las alertas SSH se registran en Wazuh.

6.5. Alertas registradas en Wazuh — victim-c

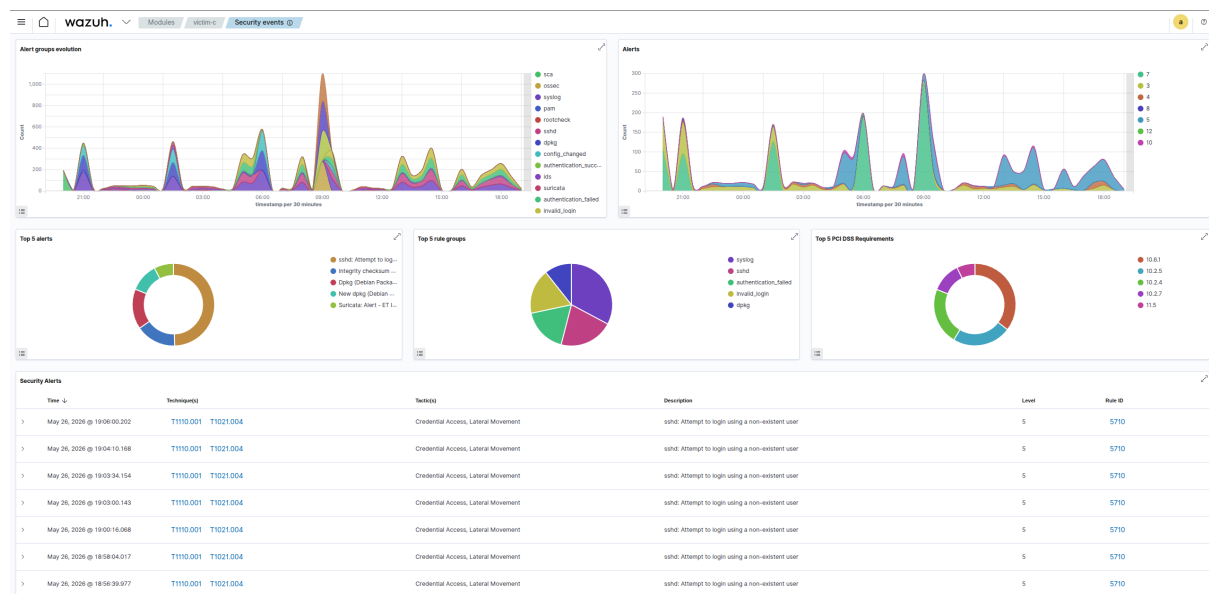


Figura 4: Security Events de Wazuh filtrado a victim-c (agent 003). *Arriba izquierda:* evolución de grupos de alertas en las últimas 24 h — el pico de `sshd` y `authentication_failed` coincide con la sesión de laboratorio. *Abajo:* lista de alertas en tiempo real; todas clasificadas como T1110.001 + T1021.004 (*Credential Access, Lateral Movement*), nivel 5, Rule 5710 (*sshd: Attempt to login using a non-existent user*).

6.6. Clasificación MITRE ATT&CK

MITRE ATT&CK (*Adversarial Tactics, Techniques and Common Knowledge*) es un marco de referencia global que cataloga tácticas y técnicas usadas por atacantes reales, organizadas en una matriz. Wazuh enriquece automáticamente cada alerta con la táctica y técnica MITRE correspondiente, permitiendo clasificar el ataque en términos estándar de la industria. Las relevantes para este experimento son:

- **TA0008 — Lateral Movement:** táctica que describe el movimiento de un adversario por los sistemas de una red comprometida.
- **T1110.001 — Brute Force: Password Guessing:** técnica de intentar credenciales comunes para autenticarse.
- **T1021.004 — Remote Services: SSH:** acceso remoto a sistemas mediante el protocolo SSH.

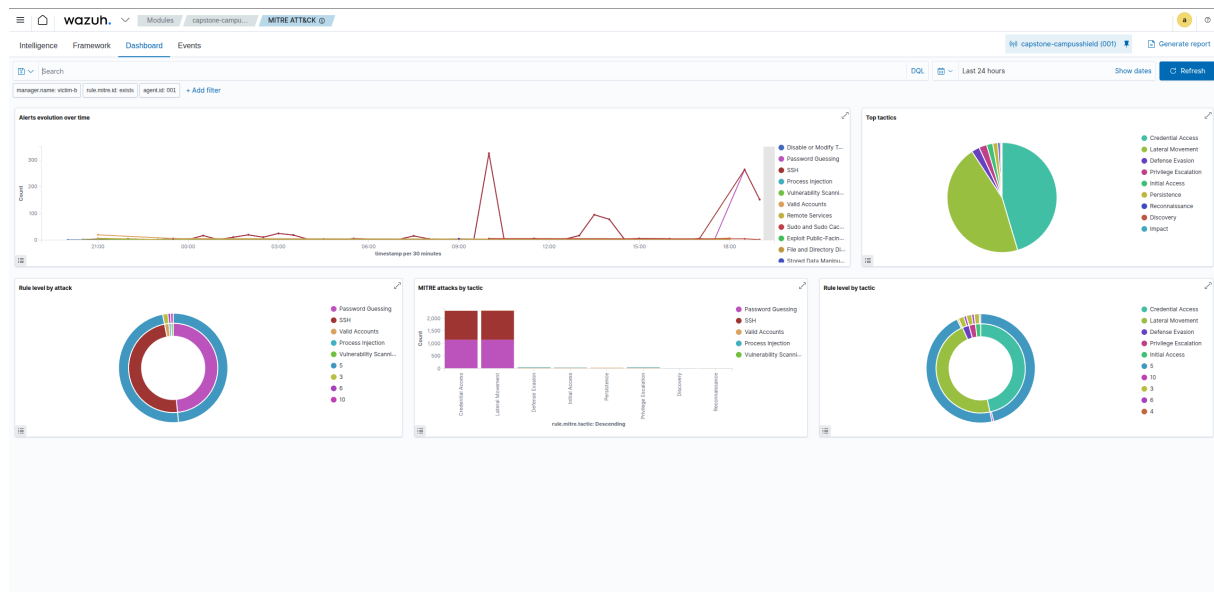


Figura 5: Dashboard MITRE ATT&CK de Wazuh para capstone (agent 001, filtrado a las últimas 24h). Las tácticas dominantes son **Credential Access** y **Lateral Movement**, con aproximadamente 2000 eventos cada una en el gráfico de barras central. El pie chart de la derecha confirma que SSH y *Password Guessing* concentran la mayoría de las técnicas detectadas, alineándose con los ataques brute-force del simulador.

7. Resultados y análisis

7.1. Evolución del campo escalar durante el experimento

Las figuras 6, 7 y 8 muestran el estado del motor en tres momentos representativos: inicio del experimento, fase estable y estado final. Cada figura contiene tres paneles: campo escalar V , gradiente ∇V , y trayectoria $\mathbf{r}(t)$.

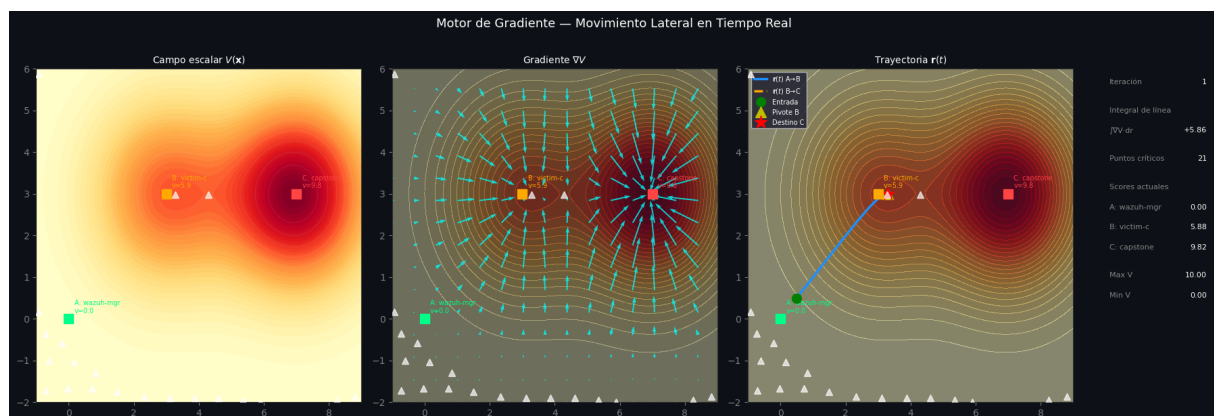


Figura 6: Iteración 1. Los nodos B (victim-c, $v = 5.88$) y C (capstone, $v = 9.82$) ya muestran picos activos porque el simulador comenzó antes de esta captura. *Panel izquierdo:* dos gaussianas bien diferenciadas con capstone claramente más intensa. *Panel central:* el campo ∇V muestra flechas que convergen hacia ambos picos; se aprecia la región de silla entre B y C donde las flechas se «abren» en direcciones opuestas. *Panel derecho:* la trayectoria en etapa A→B (azul) parte del punto de entrada (0.5, 0.5) y converge hacia victim-c. Integral de línea: +5.86. Puntos críticos detectados: 21.

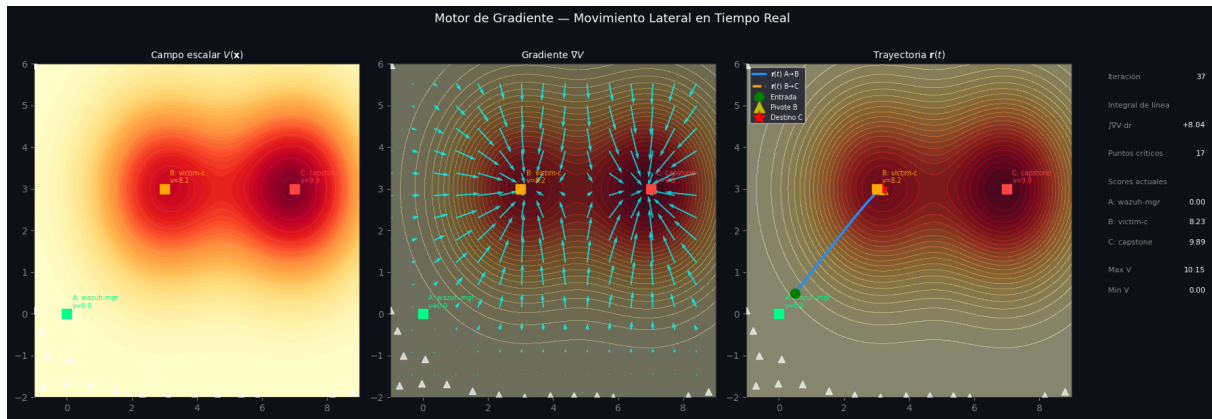


Figura 7: Iteración 37. Estado estable del experimento. Scores: $v_B = 8.23$, $v_C = 9.89$. Los dos picos gaussianos están bien desarrollados y la región de silla entre ellos es claramente visible en el panel de gradiente como zona de baja magnitud vectorial (flechas más cortas). La trayectoria A→B (azul) y B→C (naranja punteado) muestra el patrón completo de movimiento lateral. Integral de línea: +8.04.

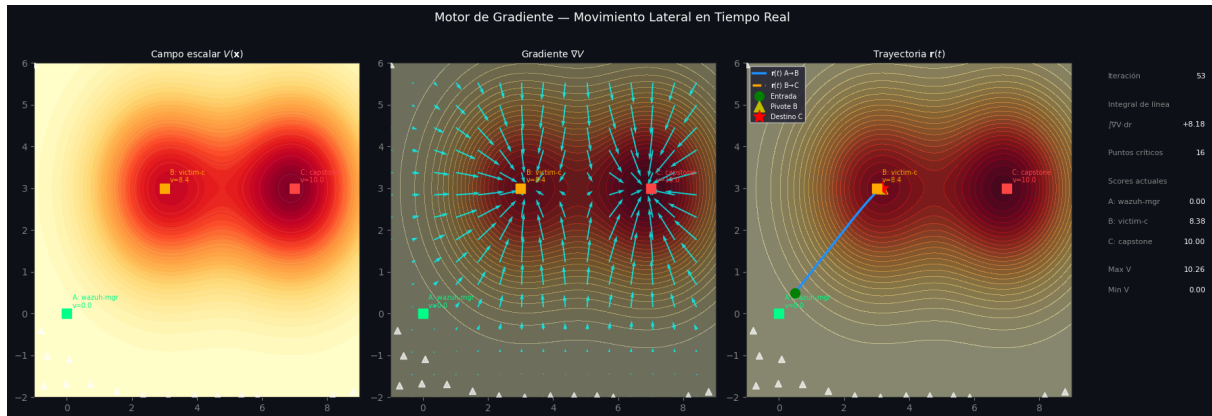


Figura 8: Iteración 53 (estado final). Capstone alcanza su score máximo ($v_C = 10.0$, saturado), mientras victim-c mantiene $v_B = 8.38$. El campo V muestra dos picos prácticamente iguales en intensidad con la silla bien definida. La integral de línea +8.18 es la más alta registrada en el experimento. Puntos críticos: 16.

7.2. Observación 1 — V responde en tiempo real

A medida que el simulador enviaba intentos SSH, los scores crecieron proporcionalmente al volumen de alertas recibidas:

Cuadro 7: Evolución de scores e integral de línea durante el experimento.

Iteración	v_A	v_B	v_C	$\int \nabla V \cdot d\mathbf{r}$
1	0.00	5.88	9.82	+5.86
9	0.00	7.39	8.90	+7.22
37	0.00	8.23	9.89	+8.04
53	0.00	8.38	10.00	+8.18

La integral creció de forma monótonica con los scores, validando empíricamente el Teorema Fundamental de Integrales de Línea: $\int \nabla V \cdot d\mathbf{r} = V(\text{destino}) - V(\text{origen})$.

7.3. Observación 2 — El saddle point es el hallazgo topológico central

En todas las figuras con dos nodos activos, el panel de gradiente muestra una región donde las flechas se «abren» en dos conjuntos opuestos. Esa región corresponde a un **punto de silla** del campo escalar: un punto crítico donde el Hessiano tiene eigenvalores de signo opuesto. Matemáticamente separa las dos cuencas de atracción del campo.

Implicación defensiva. El saddle point ($\approx (5, 3)$ entre B y C) es el cuello de botella topológico de la red. Colocar controles de acceso o honeypots en esa zona maximiza la probabilidad de interceptar al atacante en tránsito entre los dos nodos.

7.4. Observación 3 — Puntos críticos correlacionan con estructura del campo

Cuadro 8: Puntos críticos detectados según el estado del campo.

Estado del campo	Puntos críticos	Interpretación
1 nodo activo	46–94	Campo casi plano fuera del pico — artefactos
2 nodos activos	13–21	Campo bien estructurado con silla definida
3 nodos activos	5–8	Estructura muy clara, mínimos y máximos separados

7.5. Observación 4 — Tres nodos activos colapsan la integral

En las iteraciones 23–35, el wazuh-manager generó alertas propias ($v_A \approx 4$). El pico gaussiano de A «empuja» al atacante de vuelta desde el punto de entrada, haciendo que la integral caiga a valores ≈ 0.07 – 0.13 :

Cuadro 9: Iteraciones con nodo A activo.

Iter	v_A	v_B	v_C	$\int \nabla V \cdot d\mathbf{r}$
26	4.41	8.10	8.03	+0.13
33	4.09	9.04	9.56	+0.07
35	4.01	8.33	10.00	+0.09

Esto es una limitación identificada del modelo: en producción se requeriría filtrar por tipo de alerta para excluir ruido operacional del nodo de entrada.

7.6. Observación 5 — La trayectoria $A \rightarrow B \rightarrow C$ valida el movimiento lateral

La integración en dos etapas reproduce el patrón clásico de movimiento lateral de MITRE TA0008:

- **Etapa $A \rightarrow B$** (azul): el flujo gradiente desde $(0.5, 0.5)$ converge hacia victim-c, el primer máximo local accesible. Replica la fase de «pivote inicial».
- **Etapa $B \rightarrow C$** (naranja): desde el pivote en B, el flujo reanuda hacia capstone. Una vez comprometido B, el gradiente orienta al atacante hacia el activo de mayor valor.

8. Discusión: alcances, limitaciones y posibles mejoras

8.1. Alcances

- $V(x, y)$ convierte la noción abstracta de «vulnerabilidad» en un campo escalar operable con herramientas de cálculo vectorial
- ∇V formaliza la dirección óptima del ataque en cada punto
- $\mathbf{r}(t)$ describe la trayectoria como curva parametrizada con solución analítica exacta en la Fase 1
- La integral de línea cuantifica la exposición acumulada independientemente de la ruta (conservatividad verificada)
- El experimento con datos reales de Wazuh confirma que el campo responde de forma coherente a los ataques simulados

8.2. Limitaciones

Modelo analítico (Fase 1): la función V fue diseñada para satisfacer las propiedades matemáticas deseadas, no derivada de datos empíricos. Una red real es un grafo discreto con firewalls, credenciales y segmentos; el plano continuo es una idealización.

Modelo dinámico (Fase 2):

- El score basado en niveles de alerta puede ser ruidoso: alertas operacionales del nodo de entrada distorsionan el campo
- El parámetro σ requiere calibración empírica por red
- La interpolación gaussiana no respeta la topología real

8.3. Posibles mejoras

- **Grafo discreto ponderado:** reemplazar el plano continuo por un grafo con aristas ponderadas por dificultad de movimiento
- **Superficie 3D:** modelar $z = V(x, y)$ para visualizar el «relieve de vulnerabilidad» de la red
- **Barreras defensivas:** incorporar firewalls y MFA como barreras de potencial donde V tiene una discontinuidad
- **Integración con OpenCTI:** usar TTPs de MITRE ATT&CK para asignar pesos fundamentados por técnica de ataque

9. Bitácora de uso de inteligencia artificial

Herramienta utilizada

Se utilizó **Claude** (Anthropic) como asistente de inteligencia artificial a lo largo del proyecto, principalmente en dos roles: generación de código Python y contraste conceptual para definir el enfoque del modelo.

Generación de código

El motor de gradiente completo (`wazuh_client.py`, `scalar_field.py`, `gradient_flow.py`, `visualizer.py`, `main.py` y `attack_sim.py`) fue desarrollado en colaboración con Claude. El proceso consistió en describir el objetivo de cada módulo, revisar el código generado, identificar errores de ejecución y solicitar correcciones iterativas. Claude no tomó decisiones de diseño de forma autónoma: cada elección (uso de RK45, normalización logarítmica, integración en dos etapas, acceso vía SSH al archivo de alertas) fue discutida y validada antes de implementarse.

Definición del enfoque y campo de aplicación

La idea central de modelar el movimiento lateral como flujo gradiente surgió de contrastar conocimientos propios de la carrera de ingeniería y de certificaciones de ciberseguridad (familiaridad con el marco MITRE ATT&CK, con la arquitectura de SIEMs y con la etapa de movimiento lateral en ataques APT) con las sugerencias de Claude sobre cómo representar matemáticamente ese fenómeno usando cálculo vectorial.

La sesión de trabajo con Claude funcionó como un diálogo técnico: el estudiante aportó el contexto de dominio (qué es un agente Wazuh, cómo se generan alertas SSH, qué significa una táctica MITRE) y Claude aportó la formalización matemática (cómo construir V como superposición gaussiana, cómo expresar la trayectoria como solución de una EDO, cómo verificar conservatividad). El enfoque final — campo escalar de vulnerabilidad con decaimiento temporal, flujo gradiente como trayectoria del atacante, saddle point como cuello de botella defensivo — es producto de ese contraste iterativo.

Reutilización del laboratorio

La infraestructura de DigitalOcean (manager Wazuh, victim-c y capstone-campusshield) fue reutilizada de un laboratorio de **Cyber Threat Intelligence (CTI)** desarrollado previamente en otra asignatura. Dicho laboratorio ya tenía los agentes Wazuh instalados y configurados, Suricata activo con reglas Emerging Threats, y conectividad entre nodos. Para este proyecto se adaptó la infraestructura existente habilitando autenticación por contraseña en capstone y desarrollando el motor de gradiente como capa de análisis sobre los datos de alertas ya disponibles.

Lo que la IA no hizo

Claude no definió el fenómeno de interés, no eligió el campo de aplicación ni determinó qué conceptos de cálculo vectorial eran pertinentes. Tampoco ejecutó el laboratorio ni interpretó los resultados: la lectura de los scores, la identificación del saddle point y las conclusiones defensivas son producto del análisis propio contrastado con el modelo generado.

10. Conclusiones

El proyecto demuestra que el cálculo vectorial puede aplicarse de forma rigurosa a un fenómeno de ciberseguridad. El movimiento lateral de un atacante tiene una estructura matemática precisa: un campo escalar de vulnerabilidad V cuyo gradiente ∇V orienta la trayectoria hacia los activos de mayor exposición.

Los siete conceptos del curso utilizados cumplen cada uno una función interpretativa concreta dentro del modelo:

- El **gradiente** formalizó la dirección óptima del ataque
- La **curva parametrizada** $\mathbf{r}(t) = (7.4(1 - e^{-2t}), 3.25(1 - e^{-2t}))$ describió analíticamente el recorrido
- La **integral de línea** ($= 65.1$) cuantificó la exposición acumulada independientemente de la ruta
- La **conservatividad** reveló que solo importan el origen y el destino, no el camino
- El **punto crítico** con Hessiana negativa definida identificó matemáticamente la zona de mayor vulnerabilidad
- El **punto de silla** entre B y C emergió como el cuello de botella topológico donde concentrar las defensas
- La **irrotacionalidad** confirmó que no existen bucles de vulnerabilidad cerrados en el modelo

La extensión al laboratorio con Wazuh y Suricata convierte el modelo teórico en un experimento verificable. Los datos reales confirman que el campo V responde coherentemente al volumen de alertas, la trayectoria $A \rightarrow B \rightarrow C$ es reproducible, y el saddle point entre nodos es un hallazgo topológico con implicaciones defensivas concretas.

Defender la red significa **reducir las pendientes del campo** ∇V que conducen hacia los activos críticos: una idea que el cálculo vectorial hace precisa y cuantificable.

11. Referencias bibliográficas

- [1] Marsden, J. E., & Tromba, A. (2012). *Vector Calculus* (6th ed.). W. H. Freeman.
- [2] MITRE ATT&CK. *Lateral Movement, Tactic TA0008*. <https://attack.mitre.org/tactics/TA0008/>
- [3] MITRE ATT&CK. *T1110.001 — Brute Force: Password Guessing*. <https://attack.mitre.org/techniques/T1110/001/>
- [4] MITRE ATT&CK. *T1021.004 — Remote Services: SSH*. <https://attack.mitre.org/techniques/T1021/004/>
- [5] Wazuh. *Wazuh Documentation v4.7*. <https://documentation.wazuh.com/current/>
- [6] Suricata. *Suricata User Guide*. <https://docs.suricata.io/>
- [7] CISA. *StopRansomware Guide*. <https://www.cisa.gov/stopransomware/ransomware-guide>

Anexos: Evidencia fotográfica complementaria

Las siguientes capturas complementan la evidencia presentada en el cuerpo del informe. Se incluyen para completitud del registro pero no son estrictamente necesarias para seguir el argumento principal.

A.1 Dashboard global de Security Events

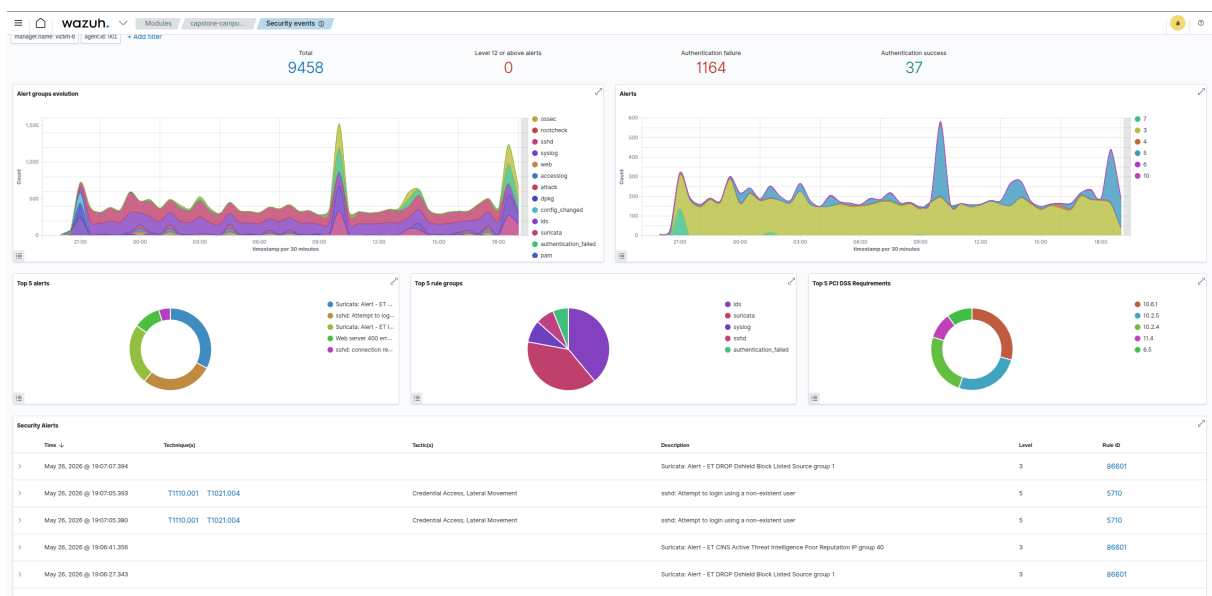


Figura 9: Security Events de capstone-campusshield (agent 001): 9 458 alertas totales, **1 164 fallos de autenticación** y 37 éxitos en las últimas 24 h. Se distinguen alertas Suricata (ET DROP, CINS) y alertas sshd. Los fallos de autenticación son las alertas que el motor consume para calcular el score de capstone.

A.2 Eventos MITRE detallados

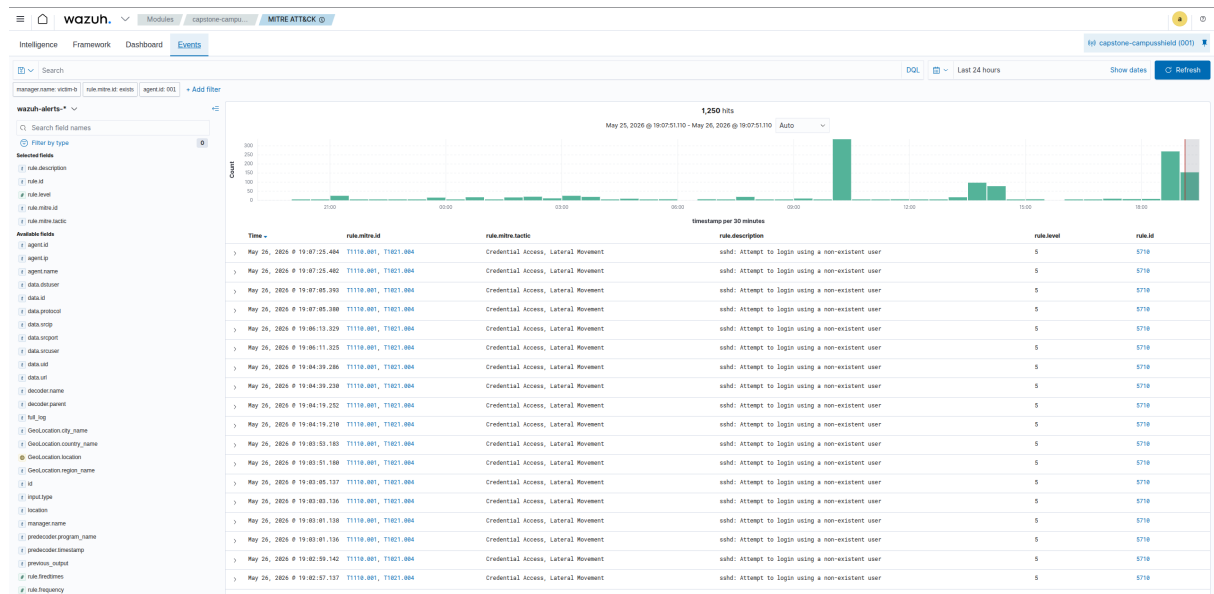


Figura 10: Vista de eventos MITRE ATT&CK para capstone: 1 250 hits en 24 h, todos clasificados como T1110.001 + T1021.004 (*Credential Access + Lateral Movement*), nivel 5, Rule 5710. La vista confirma que la totalidad de los eventos provienen de intentos SSH con usuarios inexistentes, generados por el simulador `attack_sim.py`.

A.3 Integrity Monitoring

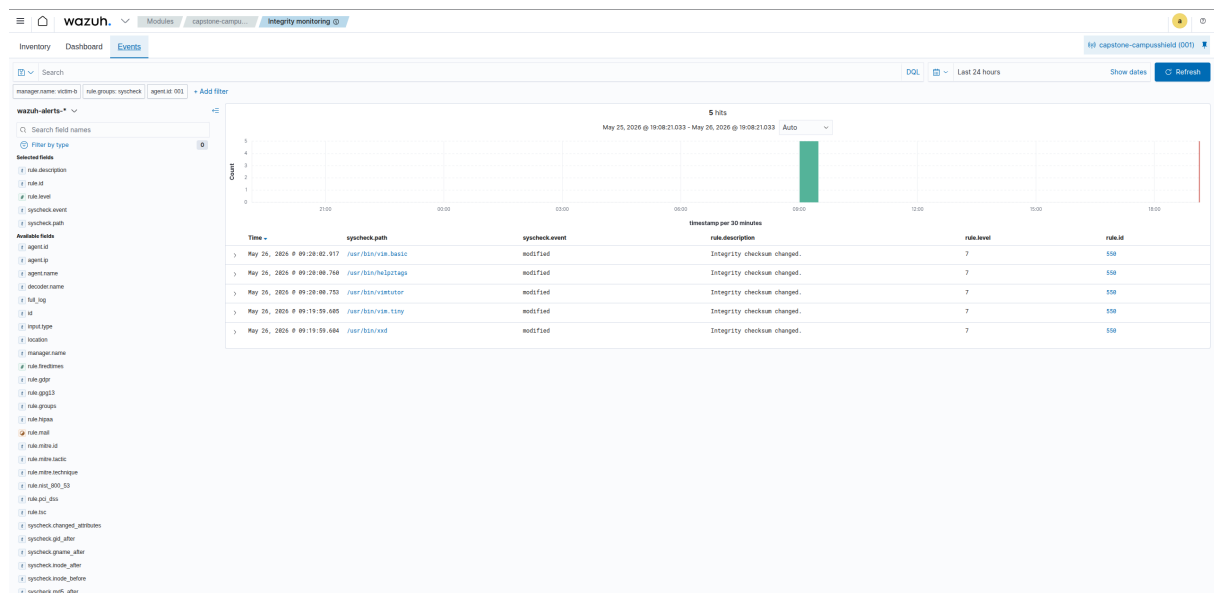


Figura 11: Integrity Monitoring (FIM) de capstone: 5 modificaciones detectadas en binarios del sistema (`/usr/bin/vim.basic`, `/usr/bin/help2tags`, `/usr/bin/vimtutor`, `/usr/bin/vim.tiny`, `/usr/bin/xxd`), Rule 550 *Integrity checksum changed*, nivel 7. Estas modificaciones corresponden a una actualización de paquetes en el nodo y no a actividad del atacante; sin embargo, ilustran la capacidad del agente Wazuh para detectar cambios en el sistema de archivos.